

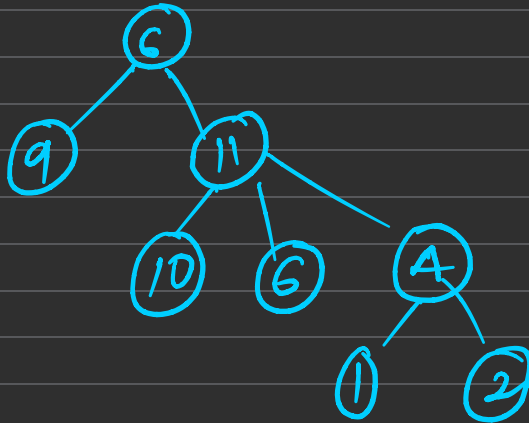
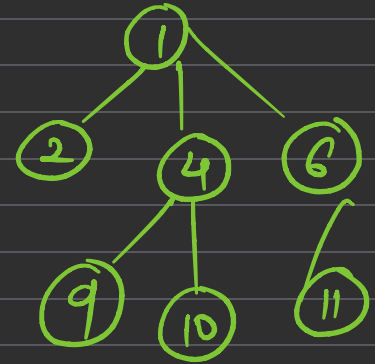
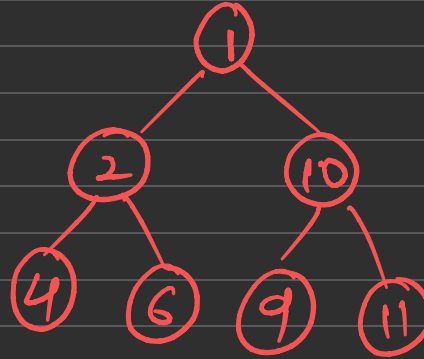
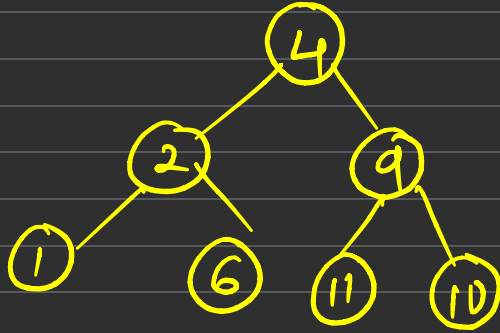
Your path is harder, maybe your calling is higher



Tree

- Non-linear Data structure
 - Parent-child Relation
 - Nodes are connected by Edges
-
- Every element is represented in form of Nodes.
 - Line which joins two Nodes $\rightarrow$ Edge

1 2 4 6 9 10 11



Terminologies of Tree

Parent Node: The node which is an immediate predecessor of a node

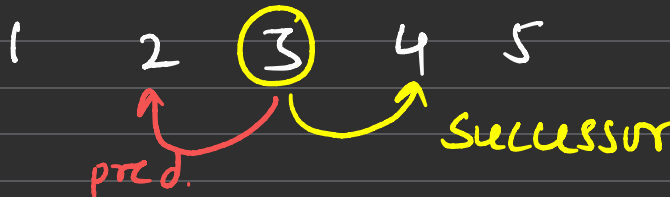
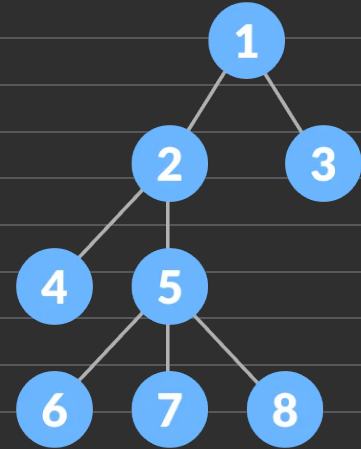
Child Node: The node which is the immediate successor of a node

Root Node: The node which does not have any Parent.

Leaf Node: The node which does not have any children.

Sibling: Children of the same parent node are called siblings.

Height of Tree: Longest path from root node to deepest leaf node.



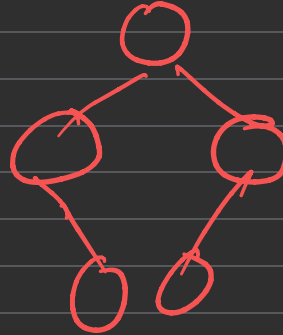
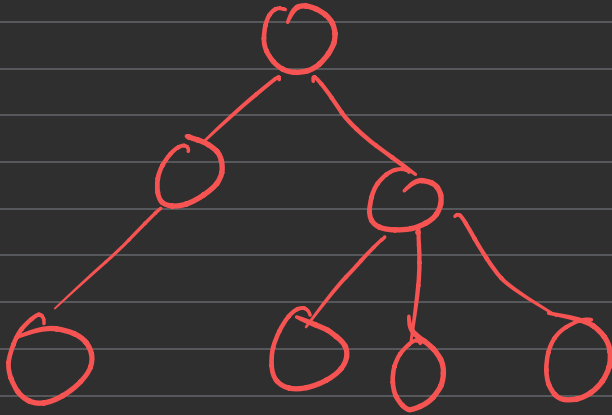
Types of Tree (BT)

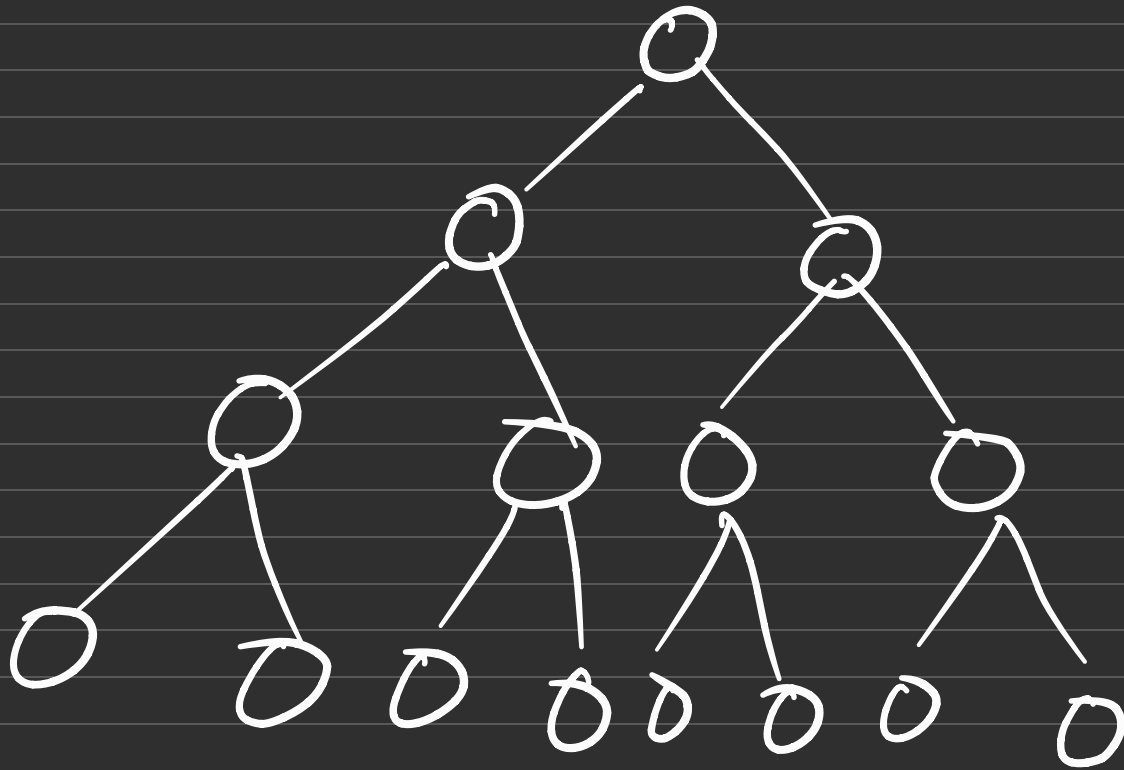
Binary Tree: In Binary Tree, each node can have almost 2 children

Complete Binary Tree: It is a binary tree, in which you can go to next level only if, one level is filled from left to right.

Full Binary Tree: It is a binary tree, in which a node can have either 2 children or no child.

- Binary Tree (0, 1, 2 children)





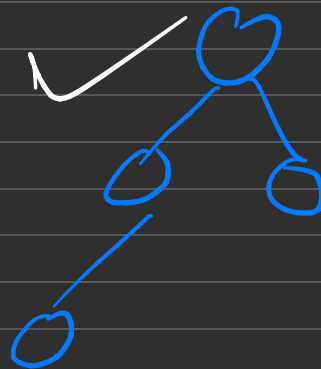
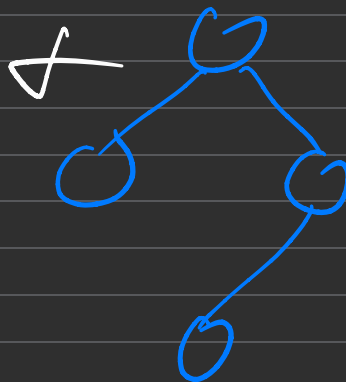
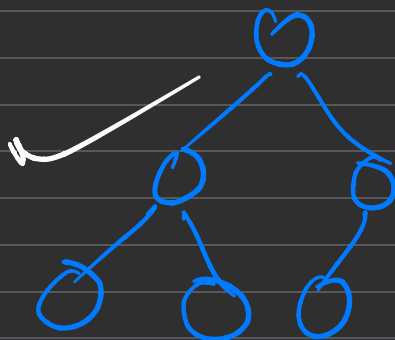
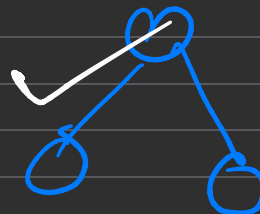
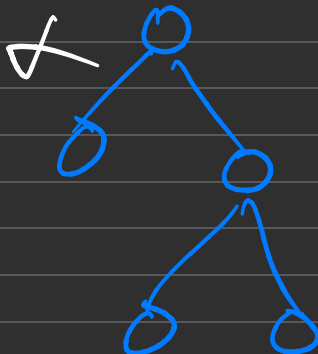
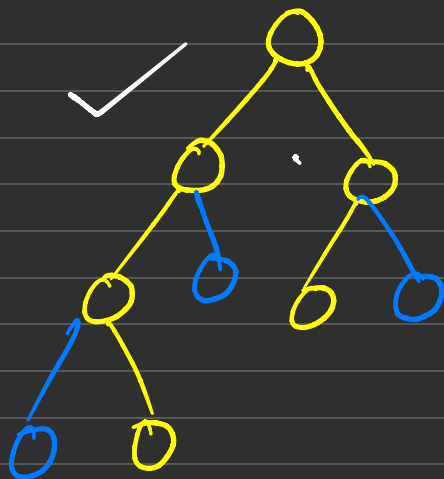
Level 0 $\overbrace{1 \quad 2^0}$

1 $\overbrace{2 \quad 2^1}$

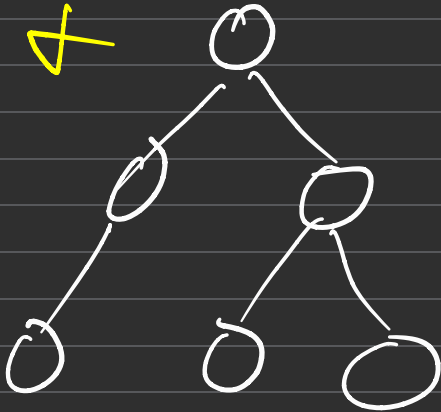
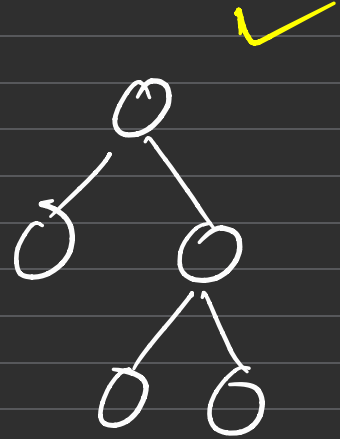
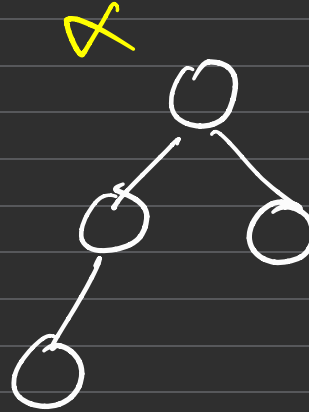
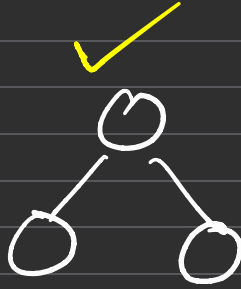
2 $\overbrace{4 \quad 2^2}$

3 $\overbrace{8 \quad 2^3}$

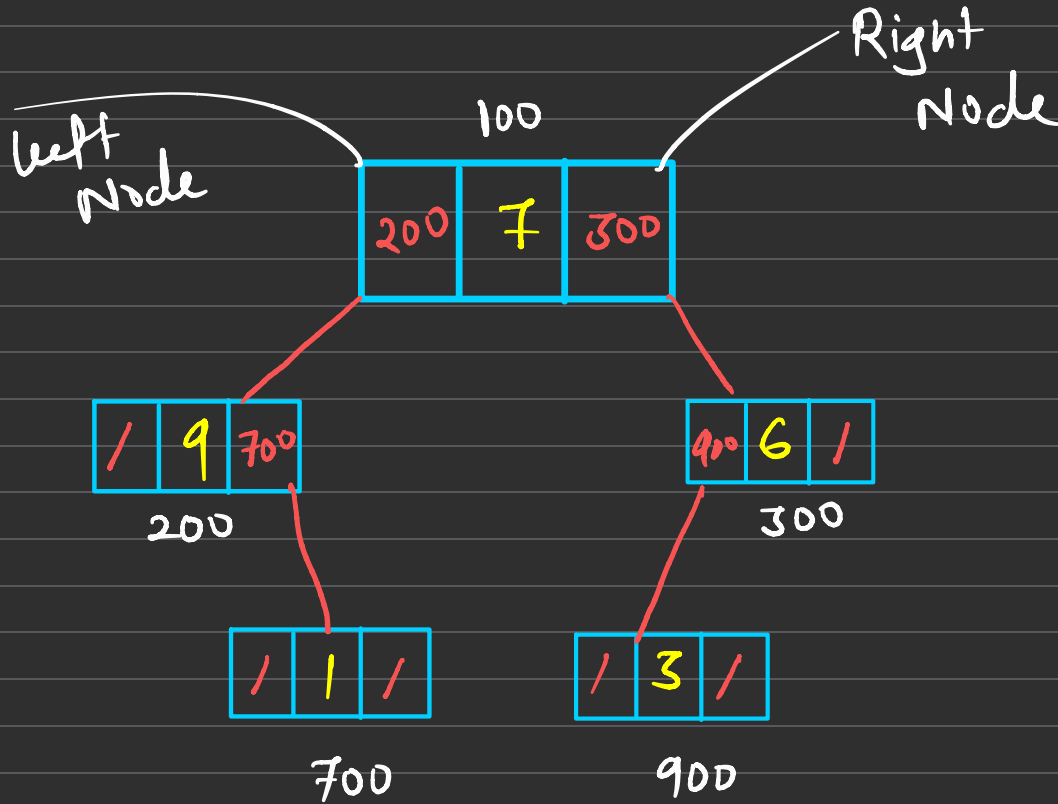
CBT



FBT



Structure of Tree



```
class Node
{
    int data
    Node * next
}
```

data	next
------	------

```
class Node
{
    int data
    Node * left
    Node * right
}
```

Tree Traversals Binary Tree

Pre-Order Traversal

Post-Order Traversal

In-Order Traversal

Level-Order Traversal

- Pre-Order Traversal

^{print}
Root Left Right

Left
→ Node

1 2 4 5 6 7 3



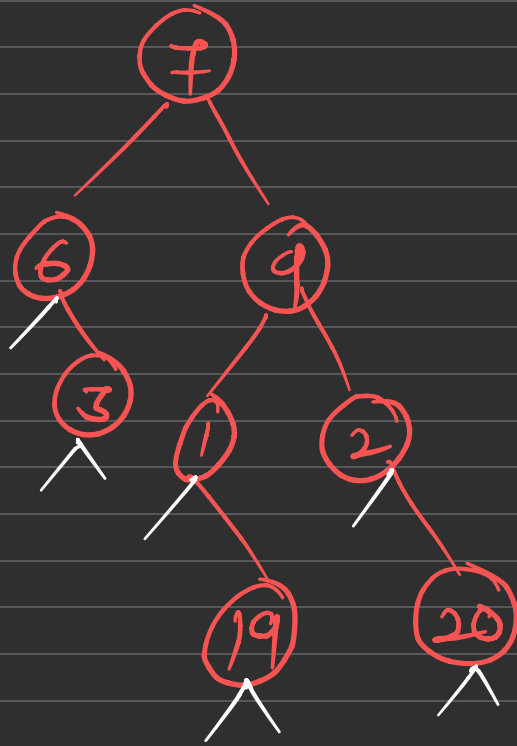
In-Order Traversal

Left **Root** Right
(print)



4 2 6 5 7 1 3





Pre-Order =

7 6 3 9 1 19 20

Inorder =

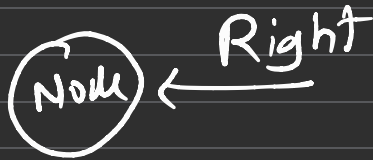
6 3 7 1 19 9 2 20

post-Order =

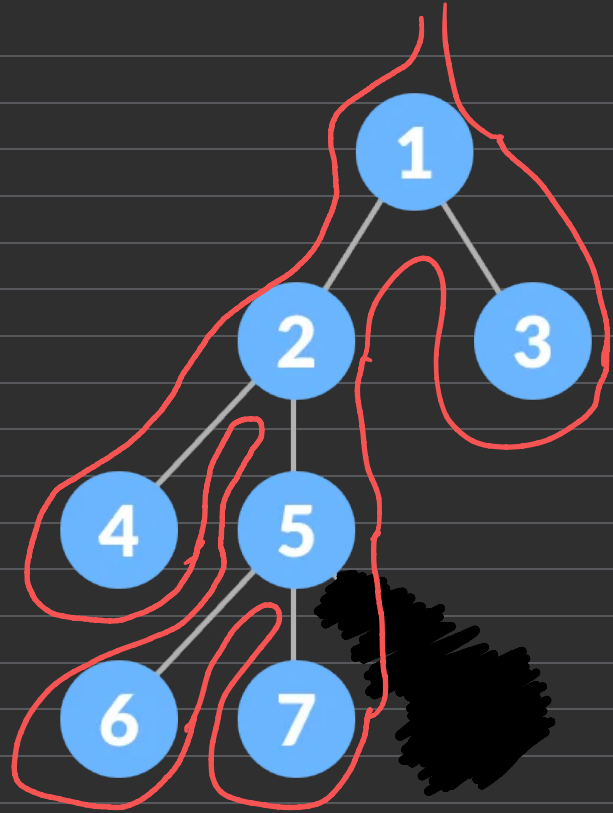
3 6 19 1 20 2 9 7

Post-Order Traversal

Left Right Root
print

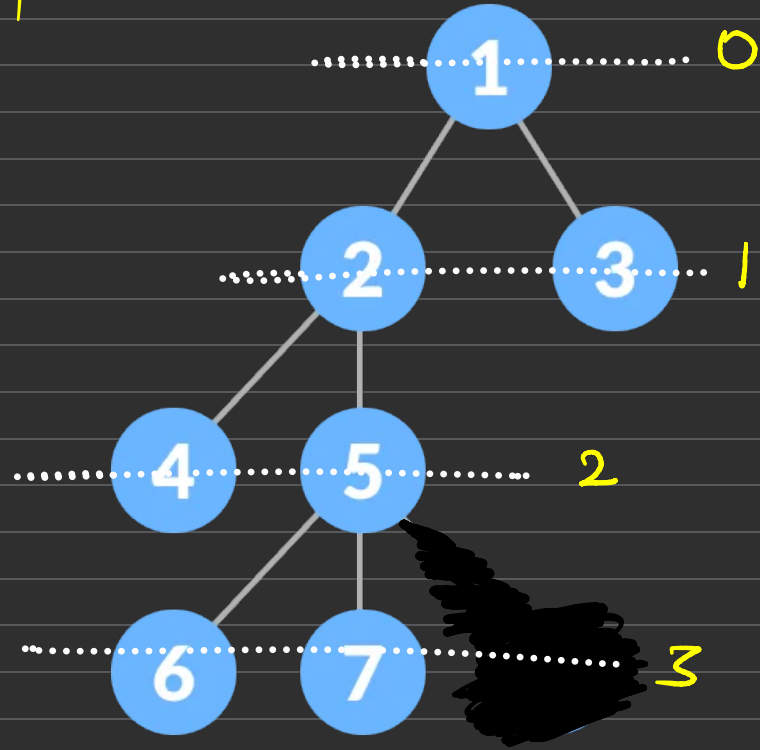


4 6 7 5 2 3 1

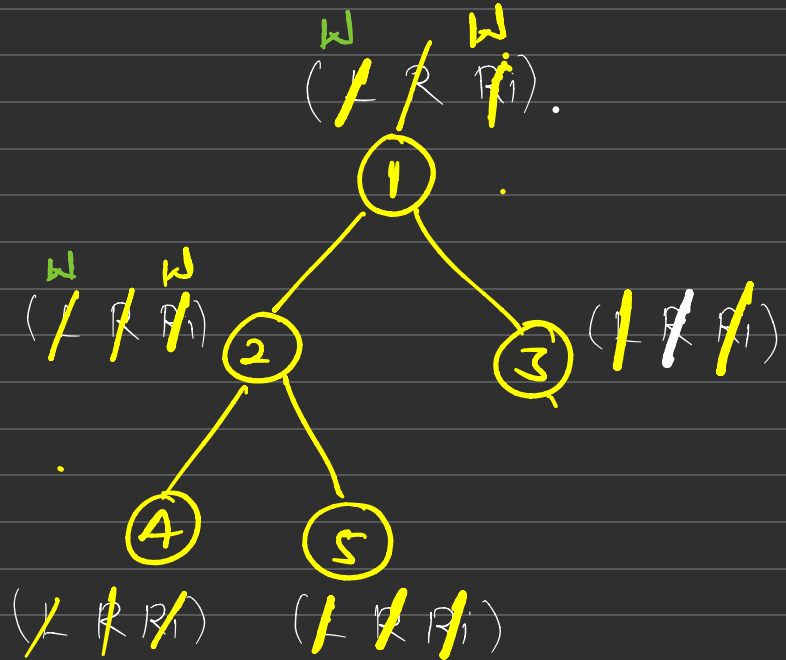
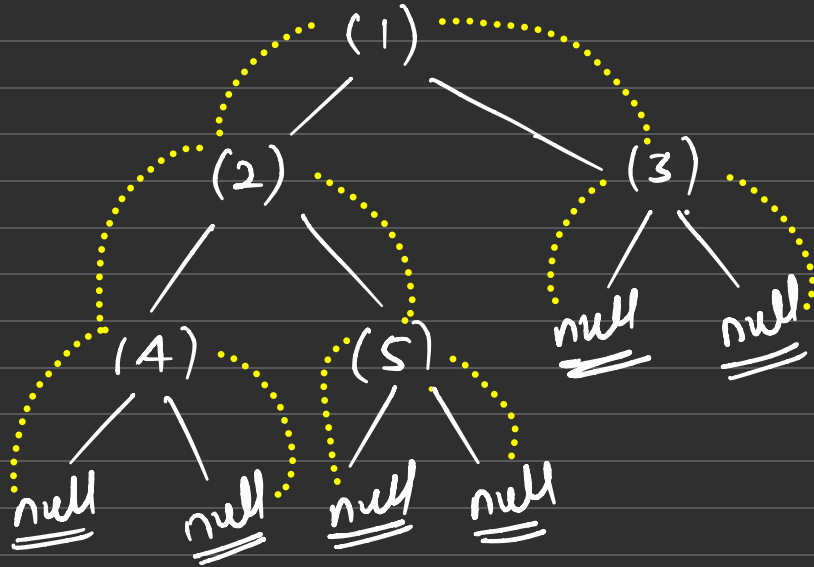


Level-Order Traversal

1 2 3 4 5 6 7



- In-Order Traversal
(Left Root Right)

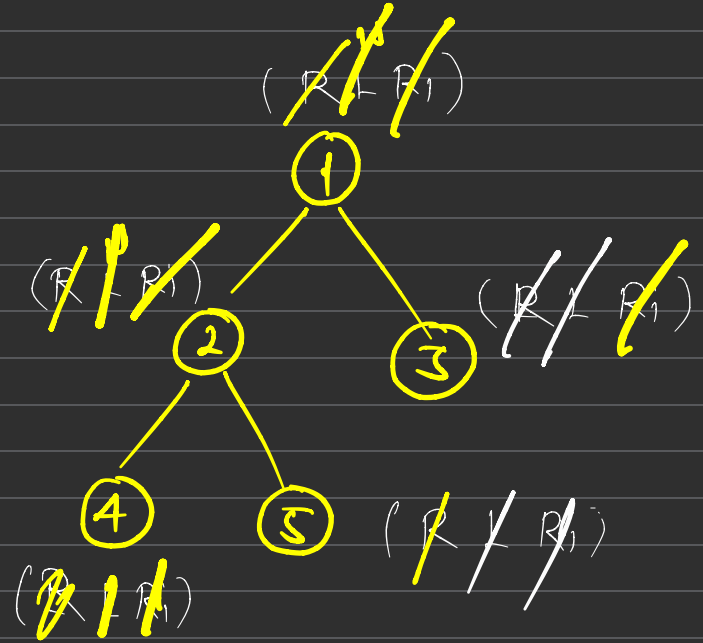
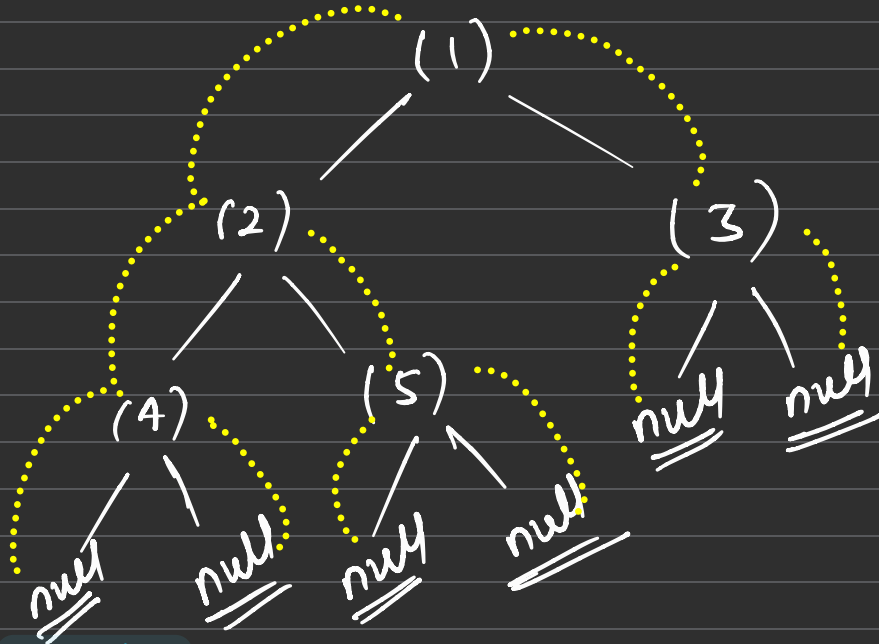


4 2 5 1 3

```
void inorder(TreeNode *root, vector<int> &ans){  
    if (root == NULL) return;  
  
    // left root(right) right  
    inorder(root -> left, ans);  
  
    ans.push_back(root -> val); // print - root  
  
    inorder(root -> right, ans);  
}
```

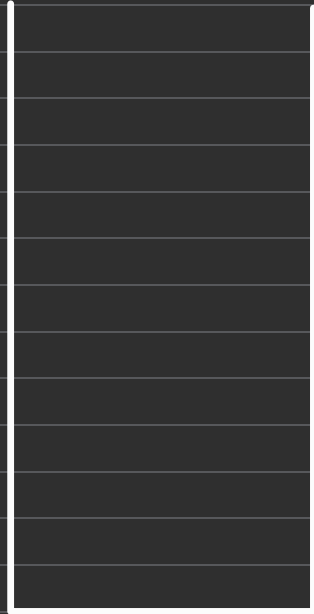
- Pre-Order Traversal

Root Left Right



1 2 4 5 3

Level-Order Traversal



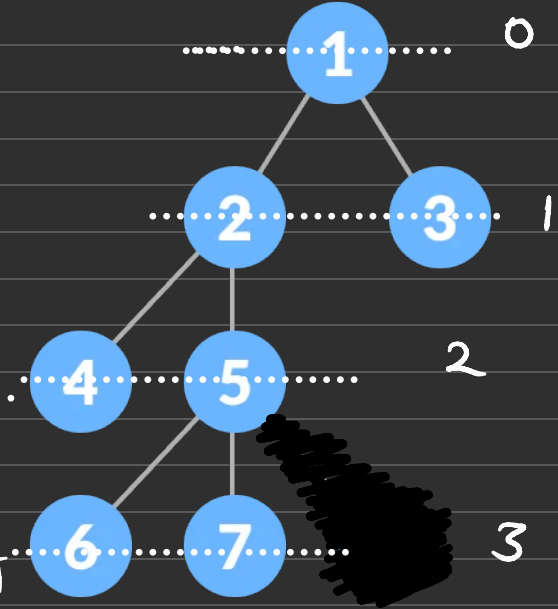
Node * node = q.front()
q.pop()

node = [7]

node->left

node->right

root



Queue

< Node * >

1 2 3 4 5 6 7

queue < Node * > q.

q.push (root)

node



while (!q.empty())

}

Node * node = q.front()

q.pop()

print (node -> data)

if (node -> left != null)

q.push (node -> left)

if (node -> right != null)

q.push (node -> right)

}

100. Same Tree

Easy

Topics

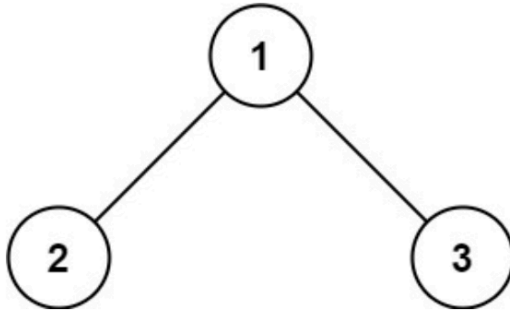
Companies

Re-do

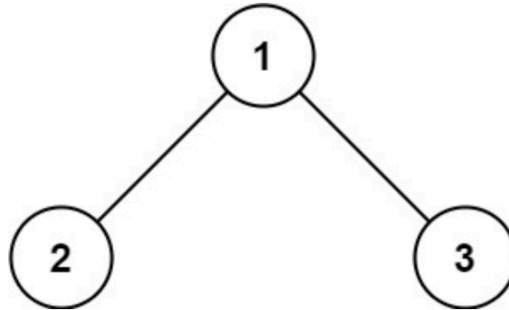
Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

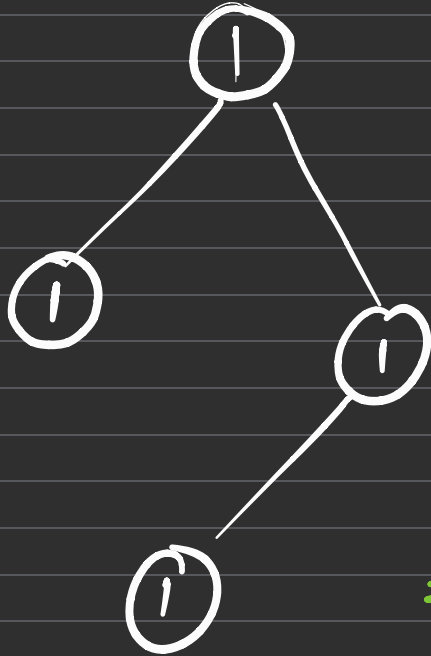
Example 1: (1 2 3)



(1 2 3)

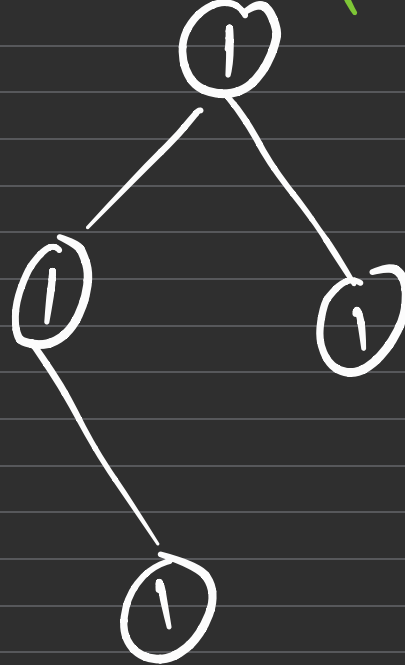


(1 1 1 1)



False

(1 1 1 1)



True X

P

Q

is Equal

Null

Null

Yes

Null

Not Null

No

Not Null

Null

No

Value

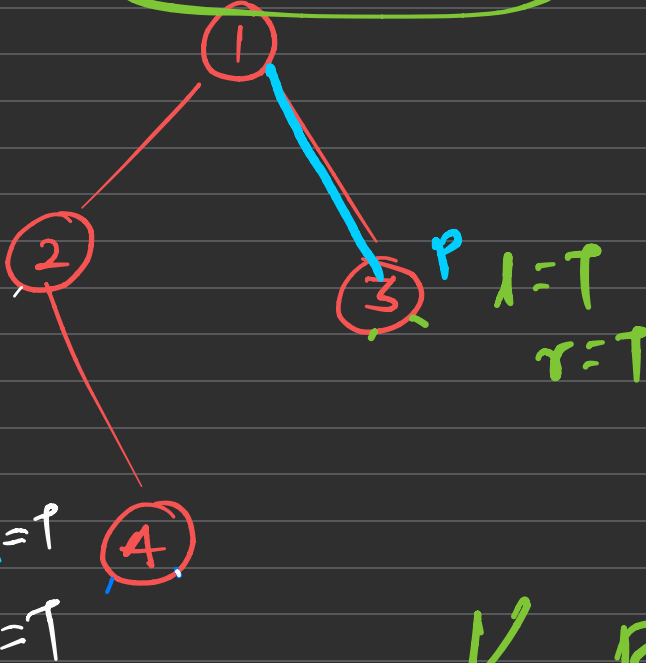
9

10

No

$l=T$ $r=T$ True

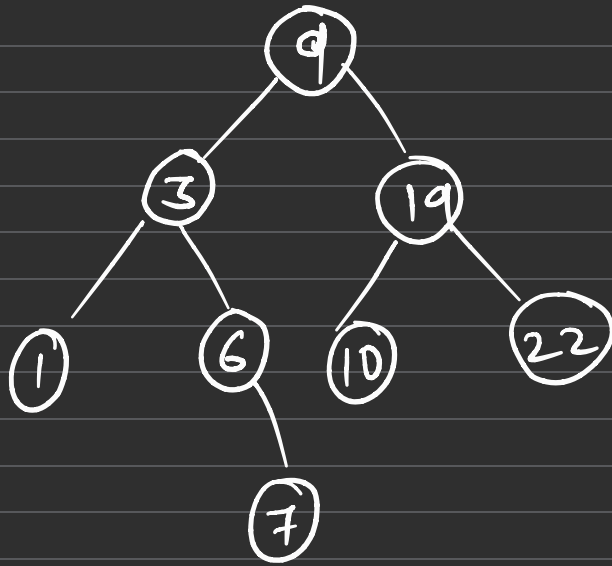
$l=True$
 $r=True$



$\swarrow$ $\searrow$ Row

Binary Search Tree

- Node which have smaller value than Root is on left side
- Node which have greater value than Root is on Right side.
- Inorder Traversal always give data in increasing.



Inorder =

1 3 6 7 9 10 19 22

701. Insert into a Binary Search Tree

Solved ✓

Medium

Topics

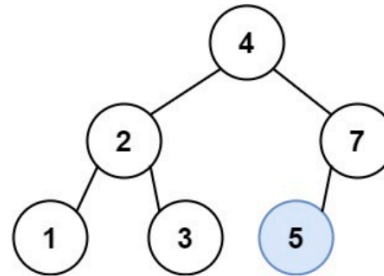
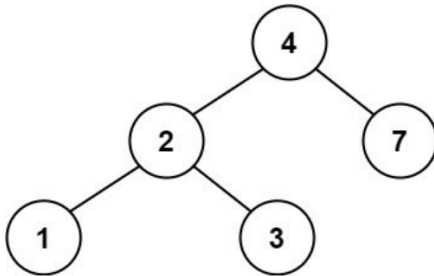
Companies

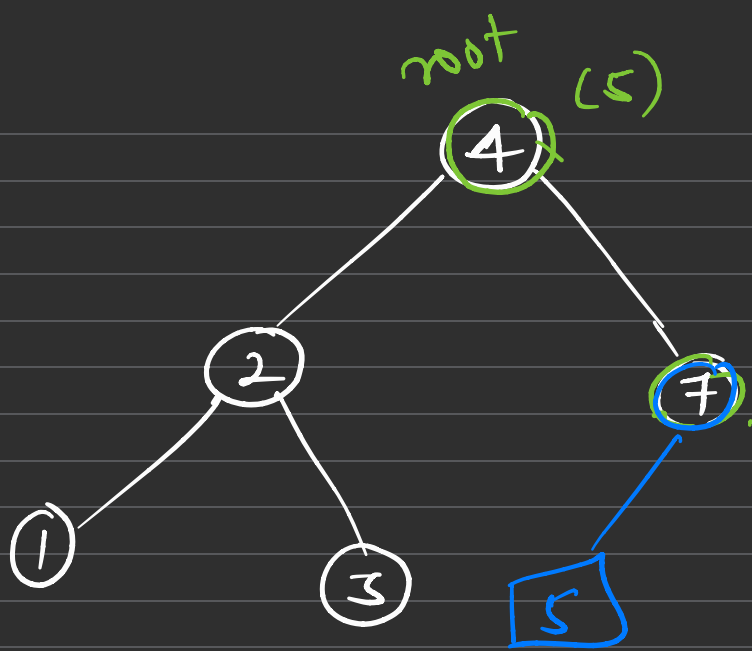
Re-do

You are given the `root` node of a binary search tree (BST) and a `value` to insert into the tree. Return *the root node of the BST after the insertion*. It is **guaranteed** that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return **any of them**.

Example 1:

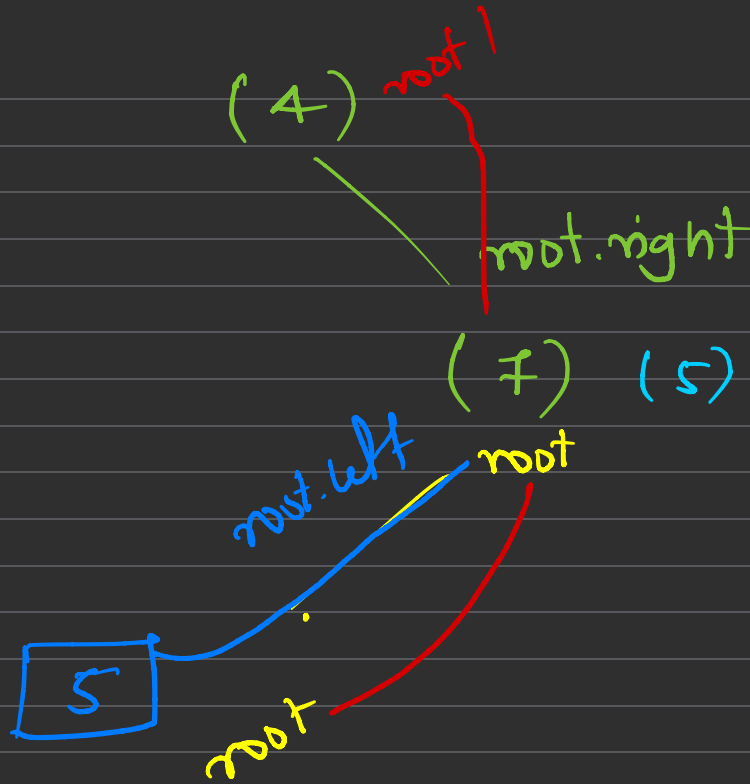




value = 5

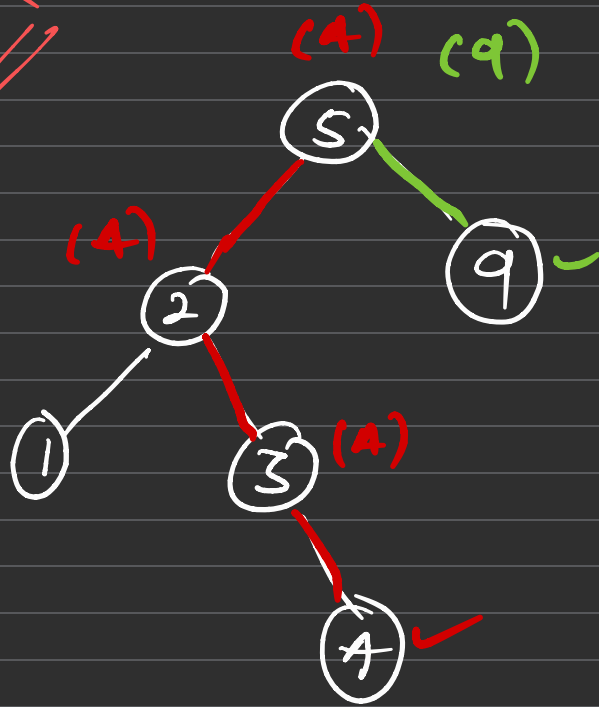
```
Node * BST (root, data)
{
    if (root == null)
    {
        Node * node = new Node(data);
        return node;
    }
    if (data > root->val)
        root->right = BST (root->right);
    else
        root->left = BST (root->left);
    return root;
}
```

⌋



AVL Tree (Balanced)

~~BS~~



(4)
snehal = 3

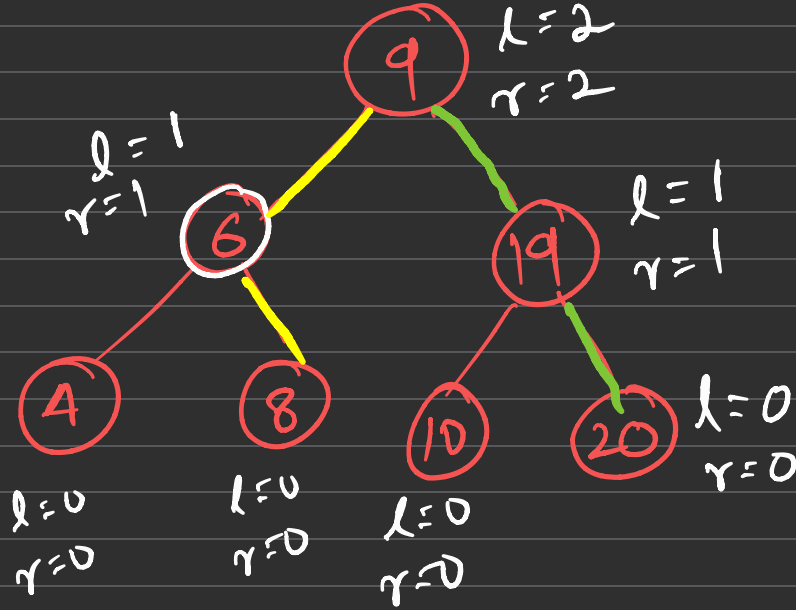
Harsh = 9 = 1

* Balanced Tree:

Each Node should have Balance fact
(0, 1, -1)

Balance = longest path — longest path
factor on left on Right

(0, 1, -1)



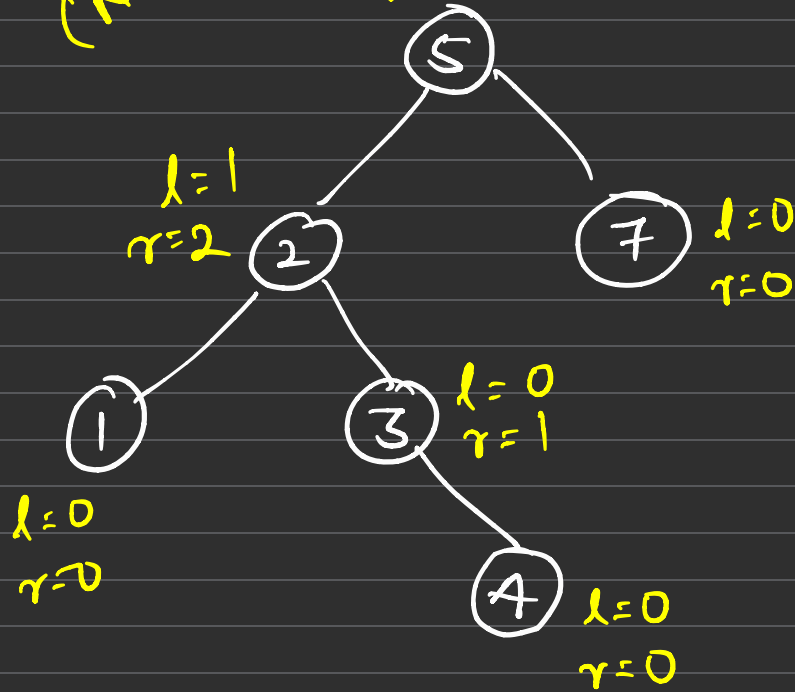
shehal = 2
(20)

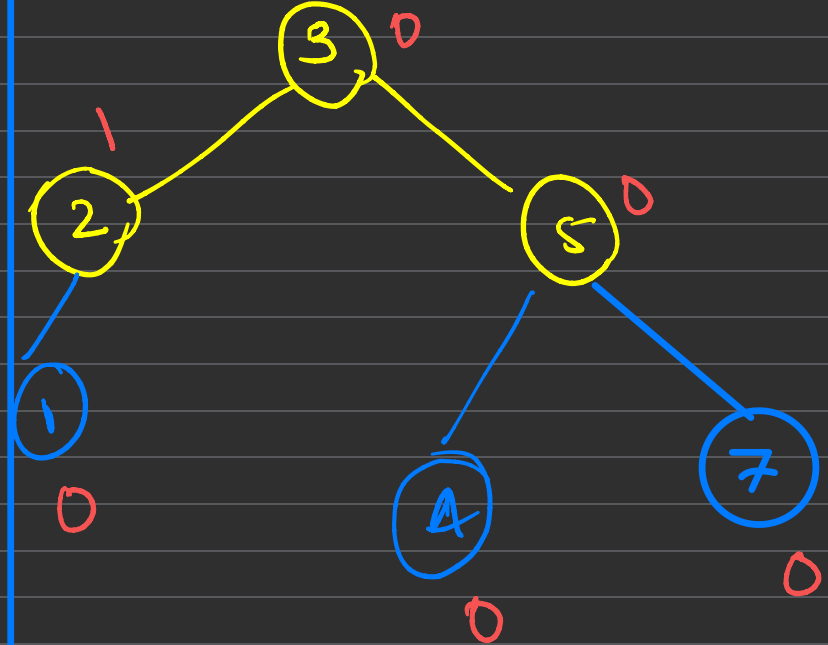
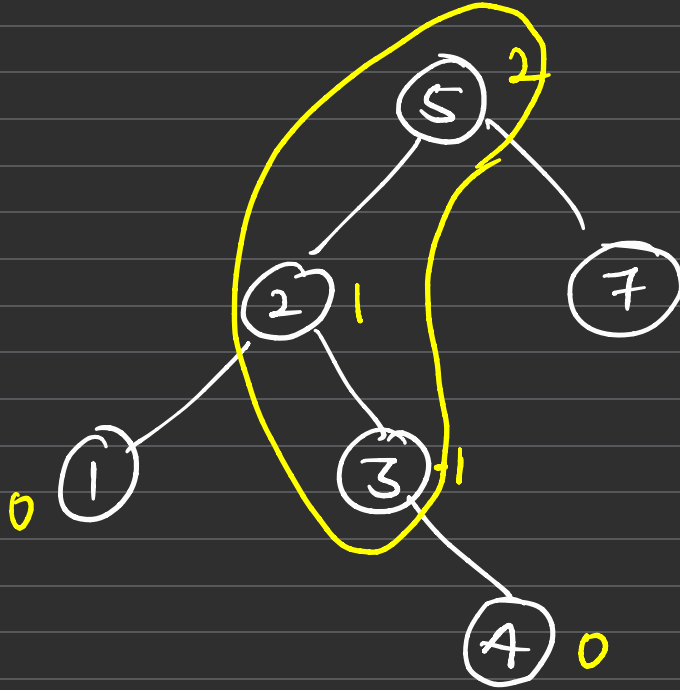
Harsh = 2
(8)

(Not Bal)

$l=3$
 $r=1$

$(0, 1, -1)$





set

map